RUHR-UNIVERSITÄT BOCHUM

FAKULTÄT FÜR INFORMATIK

Dimensionality Reduction
Project Brief

Introduction to Python

1 Introduction

Python as a programming language is often used for data analysis. Exploratory data analysis, in particular, describes the process of interactively gaining insight from data. This is very important in many areas - for instance as a research tool in almost all sciences, in business intelligence or even as a first step towards building powerful predictive analysis tools; from neural networks which detect cancer to recommender systems which suggest your next Netflix show.

The main pillars of exploratory data analysis are *dimensionality reduction*, *clustering* and *visualization*. These techniques are used to find and explore patterns in data. In this project, you will practice your Python skills by implementing two very common methods – principal component analysis (PCA) and stochastic neighbour embedding (SNE) – for dimensionality reduction. You will also visualize your results in order to compare their behaviour.

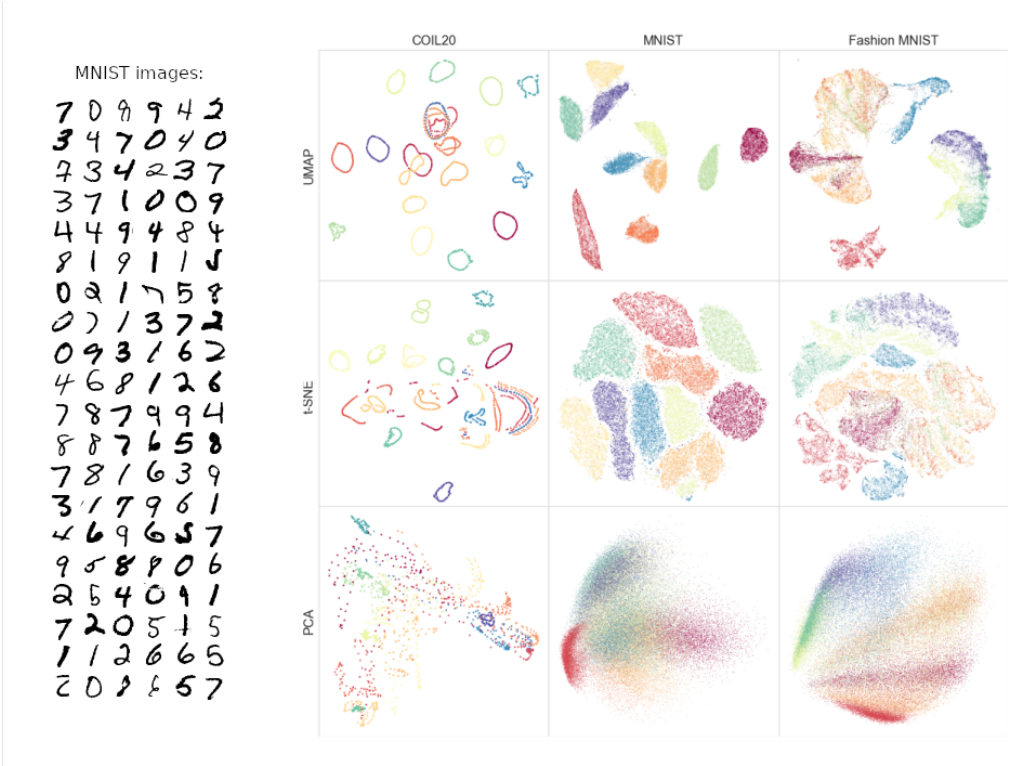

Figure 1.1: **Left:** Original 784-dimensional MNIST images of handwritten digits. **Right:** 2-dimensional results of dimensionality reduction with various algorithms (rows) on various datasets (columns). Figure adapted from MNIST-for-Numpy (GitHub) and Fig. 4 in the original UMAP paper. In this project you will apply PCA and t-SNE on the MNIST dataset (but not UMAP, since it is rather complicated).

2 Project Description

This section presents the knowledge necessary to complete the project. The next section describes how you should implement it.

The dataset you will use in this project is called MNIST. It consists of $n = 60,000$ greyscale 28×28 pixel images of handwritten digits, plus labels indicating which digit is shown in each image. If you transform an image from 2D into 1D, i.e. you just list all the individual pixels, then each image ends up being a list of $28 \cdot 28 = 784$ values. The entire dataset can thus be represented as a matrix $\mathbf{X}$ with n rows (the data points) and 784 columns (their dimensions). This is the format we will use to work with the images.

2.1 PCA

Principal component analysis is a well-known and widely used technique for linear reduction of dimensionality. It finds a linear projection of the data so that its new dimensions are uncorrelated and ordered by variance. Initially it produces as many dimensions as the original data has, but since dimensions with smaller variance have less information they can be discarded. The user has to decide how much variance/information they are willing to discard.

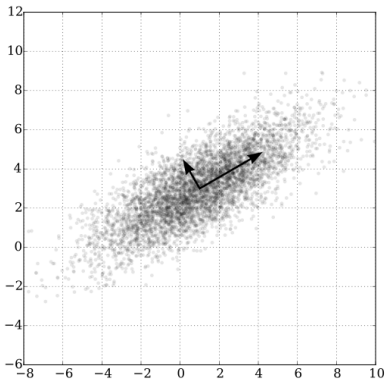

Figure 2.1: Visualisation of PCA from Wikipedia. The new dimensions of the data (visualised here with arrows) point along the direction of largest variance in the data and are uncorrelated. To reduce dimensionality, dimensions with small variance can be dropped.

One way to perform PCA is the following:

1. Center the data $\mathbf{X}$ so that the mean of each dimension across all data points is zero. Let us call the centered data $\hat{\mathbf{X}}$.

2. Calculate the covariance matrix of the centered data:

$$\mathbf{C} = \hat{\mathbf{X}}^\mathsf{T}\hat{\mathbf{X}} \tag{2.1}$$

3. Calculate the eigenvectors $\{v_i\}_{i=1\ldots784}$ and corresponding eigenvalues $\{w_i\}_{i=1\ldots784}$ of $\mathbf{C}$. The eigenvectors are our principal components and the eigenvalues quantify the squared variance of the data along each of these components. Let's write the eigenvectors as columns of a matrix $\mathbf{V}$.

4. Project the data onto the eigenvectors. For the projection, each eigenvector serves as a linear combination of the original dimensions. The projection can thus be done by multiplying the data with the eigenvectors:

$$\mathbf{Z} = \hat{\mathbf{X}}\mathbf{V} \tag{2.2}$$

   If you truncate $\mathbf{V}$ by removing columns with small eigenvalues (the last columns in $\mathbf{V}$), this is equivalent to discarding the dimensions with small variance. The equation above still works if $\mathbf{V}$ has fewer columns, the projected data $\mathbf{Z}$ will simply have fewer dimensions.

5. As a last step, you can (and most often should) scale the projected data $\mathbf{Z}$ by dividing each of its dimensions by the square root of the eigenvalue corresponding to its eigenvector. The resulting data $\hat{\mathbf{Z}}$ has unit variance in all dimensions, which is a useful property for many processing steps that might follow afterwards. PCA with this scaling (if you don't remove low-variance dimensions) is called whitening.

Usually, the number of dimensions to keep depends on amount of variance in the data you want to keep. With the eigenvalues, you can inspect how much variance each component contributes and then judge whether it's worth keeping. Dimensions with low variance are often just noise. In this project, however, we want to visualize the data. Since visualization works best in two dimensions, we will only keep the two with the largest variance.

If you want to learn more about PCA, consider reading its very good Wikipedia article or watch this video by Laurenz Wiskott.

2.2 Stochastic neighbour embedding

Stochastic neighbour embedding (SNE [link to original paper]), as opposed to PCA, is a non-linear method for dimensionality reduction. This makes it less interpretable (in PCA, dimensions of the projected data are just linear combinations of dimensions of the original data), but it also makes it more powerful.

SNE attempts to preserve neighbourhood structure by calculating a probability that any two points are nearby in the original data, and positioning the corresponding low-dimensional points according

to these probabilities. The probability values are based on the distance between points in the original data. In a low-dimensional space, an optimization method (gradient descent) is used to position all points optimally with respect to their pairwise probability of being close-by.

Mathematically, the algorithm works as follows:

1. For all pairs $\mathbf{x}_i$, $\mathbf{x}_j$ of data points, calculate their (squared and scaled) distance as

$$d_{ij} = \frac{||\mathbf{x}_i - \mathbf{x}_j||^2}{2\sigma^2} \,. \tag{2.3}$$

   The parameter σ controls the variance in P below.

2. Calculate the probability P of $\mathbf{x}_i$ and $\mathbf{x}_j$ being nearby each other as

$$p_{ij} = \frac{\exp(-d_{ij})}{\sum_{k \neq j} \exp(-d_{ik})} \,. \tag{2.4}$$

   If you plug d_{ij} into p_{ij}, you can see that this is a (scaled) Gaussian probability.

3. In low-dimensional space, we calculate the same probability but call it Q. This is because it is calculated with distances between low-dimensional points $\mathbf{z}_i$ and $\mathbf{z}_j$, rather than $\mathbf{x}_i$ and $\mathbf{x}_j$. To initialize SNE, sample as many low-dimensional points $\mathbf{z}_i$ as there are original data points randomly from a Gaussian distribution (this initializes Q as a Gaussian).

4. Eventually, we want P and Q to become the same (or at least come as close as possible). Their divergence (the opposite of similarity) is calculated as the so-called Kullback-Leibler Divergence:

$$D_{KL}(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \tag{2.5}$$

   This is our loss function. The gradient $\mathbf{g}(\mathbf{z}_i)$ of $D_{KL}(P||Q)$ with respect to the position of a low-dimensional point $\mathbf{z}_i$ is calculated as

$$\mathbf{g}(\mathbf{z}_i) = \frac{\partial D_{KL}(P||Q)}{\partial \mathbf{z}_i} = 2 \sum_j (\mathbf{z}_i - \mathbf{z}_j)(p_{ij} + p_{ji} - q_{ij} - q_{ji}) \,. \tag{2.6}$$

   By iteratively updating $\mathbf{z}_i$ in the direction of the negative gradient, we can minimize $D_{KL}(P||Q)$. This is done in the following way:

   While the loss is still decreasing noticeably, repeat the following:

   (a) Calculate the gradient $\mathbf{g}(\mathbf{z}_i)$ of the loss for all points $\mathbf{z}_i$.

   (b) Update the value of $\mathbf{z}_i$ in the direction of the gradient. A simple way to do this is

$$\mathbf{z}_i = \mathbf{z}_i - \eta \mathbf{g}(\mathbf{z}_i) \,, \tag{2.7}$$

where η is a step size. It is generally advisable to decrease the step size when approaching optimum to avoid overshooting. Better than specifying η manually, however, is to use an optimizer such as Adam for updating $\mathbf{z}_i$.

(c) For controlling the progress: Calculate the loss value with the updated $\mathbf{z}_i$ and print it out.

3 Expected Functionality

You need to solve all tasks in this section, except the bonus task.

3.1 Loading the data

1. Use mnist.py from the MNIST-for-Numpy GitHub repository to load the MNIST dataset. The procedure is described in the README of the repo.
2. Since the dataset is rather large, take a random subset of 6000 images (and their respective labels).

   Note: In step 1 of Section 3.3, you will likely run into memory problems even with 6000 images. Use your creativity to devise workarounds. But don't spend too much time on this, getting everything done even if it only works for fewer than 6000 images is better than getting a distance matrix for 6000 images but having no time to complete the other parts.

3.2 PCA

1. Implement the PCA algorithm as described above, and apply it on the subset of images so that their dimensionality is reduced to two dimensions. *Hint:* You can calculate eigenvectors and eigenvalues with numpy's `numpy.linalg.eig`.
2. Plot and describe the results. Colour the plotted low-dimensional points according to the label (shown digit) of their corresponding images. *Hint:* You can use the transparency parameter of the plotting function to make the density of point clouds more visible. Are the results what you expected?

3.3 SNE

1. Implement a function to calculate distances d and to calculate probabilities P. You can either do this element-wise (d_{ij} and p_{ij}) or for the full matrices across all indices i and j. If you want to practice your numpy skills, the latter is recommended.

   Each data point $\mathbf{x}_i$ obviously has a distance to itself of $d_{ii} = 0$. This in turn means that the probabilities will be dominated by the terms p_{ii}, which you do not want. To avoid this, you

can first exponentiate the distances and then subsequently set all terms $exp(-d_{ii})$ to zero before computing and dividing by the denominator in (2.4).

You can use the same function for calculating both P and Q. The value of σ^2 needs to be a parameter of this function. When calling the function later, you can set σ^2 to 1000^2 for P and to 1 for Q, because of its lower dimensionality.

2. Implement a function to calculate the loss.

   Note that the matrices P and Q should have zero diagonals, which means Numpy will run into issues when calculating $log(\frac{p_{ii}}{q_{ii}})$. This can be dealt with by subsequently resetting these values.

3. Implement a function to calculate the gradient.

4. Initialize the positions $\mathbf{z}_{i=1...n}$ of all 6000 low-dimensional points. Make them two-dimensional and use $\mu = 0$, $\sigma = 0.1$ for their initial distribution.

5. Repeatedly perform gradient descent until it converges. Don't forget to print out the loss value at each step! Think of a suitable condition to check for convergence within the loop.

   To perform gradient descent, use the Adam optimizer. The website explaining Adam contains this (incomplete) code-snippet which you can use. Their code contains a small bug: Instead of t you should use $t + 1$ to calculate *m_hat* and *v_hat* to avoid division by 0. The parameter w in the code correponds to our set of $\mathbf{z}_i$. β_1, β_2, ϵ and *step_size* are hyperparameters which you can set to 0.9, 0.999, 10^{-8} and 0.1 respectively. Initialize the first m and the first v to 0. The *step_size* controls the speed of convergence, feel free to explore different values of it.

6. Plot the points, again coloured by label, and describe the results. How do these results compare to those from PCA?

3.4 t-SNE from scikit-learn

There is a more modern version of SNE called t-SNE. It differs in that it uses the Student-t distribution instead of a Gaussian distribution to encode the probability of points being nearby each other in low-dimensional space. There are also some additional small tricks in t-SNE which improve the dimensionality reduction compared to SNE. Here, we will use the t-SNE class implemented in the machine learning package scikit-learn.

1. Read the documentation of t-SNE.

2. Install the scikit-learn Python package if you don't have it already (*Hint:* You can check this by trying to import it)

3. Perform t-SNE on the 6000 data points using the scikit-learn class.

4. Plot the resulting low-dimensional points, again coloured by label. Describe the results. How do they compare to the results from SNE?

3.5 Bonus: Visualizing more dimensions

This last task is just a bonus. Even if you don't do it, you can still get the best grade. It will however be fun and interesting to do if you developed an interest in dimensionality reduction during the projects!

Perform the dimensionality reduction with all methods from above again, but this time make the low-dimensional points have more than two dimensions (three, four...). Can you still find ways to visually present the low-dimensional points? (*Hint:* You could for example use more than one plot, each presenting different aspects of the same dataset.) Can you still use those visualizations to compare the methods?